

Chapter 8: Stochastic Methods

Algorithms for Optimization, 2nd ed. 2026

Kochenderfer & Wheeler

Lecture Notes

2026

Chapter Outline

- 1 Introduction to Stochastic Methods
- 2 8.1 Noisy Descent
- 3 8.2 Mesh Adaptive Direct Search (MADS)
- 4 8.3 Memory-Efficient Zeroth-Order Optimization (MeZO)
- 5 8.4 Simulated Annealing
- 6 8.5 Cross-Entropy Method
- 7 8.6 Proposal Distribution Descent
- 8 8.7 Information-Geometric Optimization (IGO)
- 9 8.8 Covariance Matrix Adaptation (CMA-ES)
- 10 Method Comparison
- 11 8.9–8.10 Summary & Exercises
- 12 Conclusion

Why Stochastic Methods? I

Deterministic optimisation methods such as gradient descent follow a fixed, reproducible path through the design space. This predictability is useful, but it carries a serious weakness: the method can become permanently trapped in a *local minimum* — a point that looks like a bottom when viewed from nearby, but is not the globally lowest point. Stochastic (randomly driven) methods address this weakness by introducing controlled randomness into the search, allowing the algorithm to jump away from local traps and explore the design space more broadly.

Why Stochastic Methods? II

Four Core Reasons to Use Stochastic Methods

- 1 **Escaping local minima.** A deterministic gradient method, once it descends into a local minimum, has no mechanism to leave it. A stochastic method can occasionally accept or generate moves that go *uphill*, giving the algorithm a chance to find a deeper valley elsewhere.
- 2 **Handling noisy evaluations.** In many practical problems — training a neural network on mini-batches of data, or evaluating a design via a noisy computer simulation — the objective function $f(\mathbf{x})$ cannot be computed exactly. Stochastic methods are designed to work with noisy estimates of f or its gradient.
- 3 **Increasing coverage of the design space.** Random sampling strategies explore regions that a purely gradient-following path would never visit, which is critical when the global minimum is far from the starting point.
- 4 **Memory efficiency at large scale.** Computing and storing the full gradient of a model with billions of parameters (such as a large language model) requires enormous memory. Zeroth-order stochastic methods estimate gradient information from only a few function evaluations, requiring $O(1)$ extra memory.

Why Stochastic Methods? III

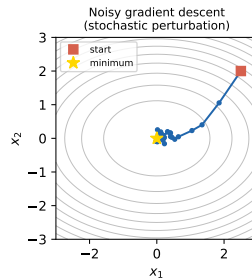
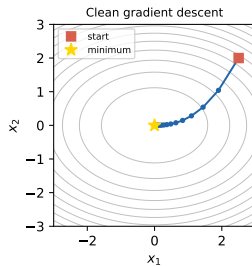
Stochastic vs. Deterministic: A Quick Comparison

Property	Deterministic	Stochastic
Trapped in local minima	Yes, permanently	Can escape
Reproducibility	Fully reproducible	Random (fix seed for reproducibility)
Memory usage	$O(n)$ for gradient	$O(1)$ to $O(n^2)$ depending on method
Gradient required	Often yes	Not always

The figure on the right illustrates the key intuition: on a quadratic bowl function, a clean gradient descent follows a smooth, straight path to the minimum, while a noisy gradient descent follows a jagged path that can escape narrow troughs that would otherwise trap a deterministic method.

Why Stochastic Methods? IV

Effect of Noise on Gradient Descent



Why Stochastic Methods? V

Clean versus noisy gradient descent on a quadratic bowl. The stochastic path is jagged but can escape narrow valleys that trap the deterministic method.

Key Takeaway

Stochastic methods trade away reproducibility and smooth convergence in exchange for the ability to explore broadly and escape local optima — a worthwhile trade-off for many real-world optimisation problems.

8.1 Noisy Descent — The Core Idea I

The simplest way to introduce randomness into gradient descent is to *corrupt the gradient* with random noise at each step. Rather than moving in the exact direction of steepest descent, we move in a direction that is close to that direction but perturbed by a random vector. This small perturbation is enough to prevent the algorithm from getting permanently stuck, while still making overall progress toward a minimum.

The Stochastic Gradient Descent (SGD) Update Rule

At each iteration k (where k is the iteration counter, starting from 1), we update our current point $\mathbf{x}^{(k)}$ (read as “ $\mathbf{x}$ superscript k ”, meaning the value of $\mathbf{x}$ at iteration number k) as follows:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha^{(k)} \tilde{\nabla} f(\mathbf{x}^{(k)})$$

8.1 Noisy Descent — The Core Idea II

Notation

- $\mathbf{x}^{(k)}$ is our current estimate of the minimiser, a vector in $\mathbb{R}^n$ (real n -dimensional space).
- $\alpha^{(k)}$ (alpha, superscript k) is the *step size* (also called the learning rate) at iteration k — it controls how large a step we take. A larger α means a bigger move.
- $\tilde{\nabla}f(\mathbf{x}^{(k)})$ (nabla-tilde f , read “noisy gradient of f at $\mathbf{x}^{(k)}$ ”) is a random approximation to the true gradient ∇f (nabla f , the vector of partial derivatives that points in the direction of steepest ascent).
- The minus sign means we move *opposite* to the gradient — downhill, not uphill.

The noisy gradient is defined as:

$$\tilde{\nabla}f(\mathbf{x}) = \nabla f(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

Here ϵ (epsilon) is a random noise vector drawn from a multivariate Gaussian (normal) distribution with mean zero (the zero vector $\mathbf{0}$) and covariance $\sigma^2 \mathbf{I}$ (sigma squared times the identity matrix $\mathbf{I}$). The parameter σ (sigma) controls the standard deviation — how large the random perturbations are. Larger σ means more noise and more exploration.

8.1 Noisy Descent — Python Implementation

```
1 import numpy as np
2
3 def noisy_descent(f, grad_f, x0, alpha=lambda k: 1.0/k,
4                  sigma=0.1, max_iter=1000, tol=1e-5):
5     """
6     f          : objective function    $\mathbb{R}^n \rightarrow \mathbb{R}$ 
7     grad_f     : gradient function    $\mathbb{R}^n \rightarrow \mathbb{R}^n$ 
8     x0         : initial point
9     alpha      : step-size schedule   $k \rightarrow \text{float}$  (default:  $1/k$ )
10    sigma      : noise standard deviation (controls exploration)
11    """
12    x = np.array(x0, dtype=float)
13    for k in range(1, max_iter + 1):
14        g = grad_f(x) # true gradient
15        noise = np.random.randn(len(x)) * sigma # random perturbation
16        g_hat = g + noise # noisy gradient
17        x = x - alpha(k) * g_hat # SGD update
18        if np.linalg.norm(alpha(k) * g_hat) < tol:
19            break
20    return x
21
22 # Example: minimize  $f(x) = x_1^2 + 2x_2^2$  from  $x_0 = [3, 3]$ 
23 f = lambda x: x[0]**2 + 2*x[1]**2
24 grad_f = lambda x: np.array([2*x[0], 4*x[1]])
25 x_opt = noisy_descent(f, grad_f, [3.0, 3.0], sigma=0.5)
26 print(f"Optimum: {x_opt}") # approx [0, 0]
```

8.2 Mesh Adaptive Direct Search (MADS) I

Mesh Adaptive Direct Search (MADS) is a *zeroth-order* method, meaning it requires only function values $f(\mathbf{x})$ — no gradients. This makes it ideal for *black-box* problems where the objective is computed by a simulation or a legacy code that provides no derivative information.

The core idea is to systematically probe a set of search directions around the current point. If any direction produces an improvement, we move there and expand our step size; if none do, we shrink the step size and try again with a finer mesh. The randomness enters through how the search directions are generated at each iteration.

What is a Positive Spanning Set?

A set of vectors $\{\mathbf{d}^{(1)}, \dots, \mathbf{d}^{(n+1)}\}$ is called a *positive spanning set* if every direction in $\mathbb{R}^n$ can be written as a *non-negative* combination of these vectors. Intuitively, the vectors “point in all directions” — they collectively cover the entire space, so no descent direction can be missed by the search.

For $n = 2$ (two-dimensional problems), a positive spanning set consists of at least 3 vectors; MADS uses $n + 1 = 3$ vectors in 2D and $n + 1$ vectors in general.

8.2 Mesh Adaptive Direct Search (MADS) II

Algorithm: Generating a Random Positive Spanning Set

MADS uses a random lower-triangular matrix to generate its search directions. The procedure is:

- 1 Compute the mesh granularity parameter δ (delta, a positive integer) from the current step size α (alpha):

$$\delta = \left\lfloor \frac{1}{\sqrt{\alpha}} \right\rfloor$$

Here $\lfloor \cdot \rfloor$ (floor) means round down to the nearest integer. As α (alpha) decreases, δ increases, making the search directions more varied and the mesh finer.

- 2 Build a random lower-triangular matrix $\mathbf{L}$ (capital L) of size $n \times n$:
 - The diagonal entries are randomly $+1$ or -1 .
 - The below-diagonal entries are random integers in $\{-\delta + 1, \dots, \delta - 1\}$.
 - Upper-triangular entries are zero.
- 3 Randomly permute the rows and columns of $\mathbf{L}$ to obtain $\mathbf{D}$ (capital D). This permutation ensures that no single coordinate is always probed first, making the search direction selection unbiased.
- 4 Add one extra direction $\mathbf{d}^{(n+1)} = -\sum_{i=1}^n \mathbf{d}^{(i)}$, which is the negative sum of all previous directions. This extra direction guarantees the positive spanning property.

8.2 MADS — Python Implementation

```
1 import numpy as np
2
3 def rand_positive_spanning_set(alpha, n):
4     """Algorithm 8.2: random positive spanning set of n+1 directions."""
5     delta = int(round(1 / np.sqrt(alpha)))
6     L = np.diag(np.random.choice([-1, 1], n))           # diagonal +/-1
7     for i in range(1, n):
8         for j in range(i):
9             L[i, j] = np.random.randint(-delta+1, delta) # subdiagonal
10    perm_r = np.random.permutation(n)
11    perm_c = np.random.permutation(n)
12    D = L[perm_r, :][:, perm_c]
13    extra = -D.sum(axis=1, keepdims=True)                # -(sum of columns)
14    return np.hstack([D, extra]).T                       # shape (n+1, n)
15
16 def mesh_adaptive_direct_search(f, x, eps=1e-4, alpha_init=1.0):
17     """Algorithm 8.3: MADS."""
18     alpha, y, n = alpha_init, f(x), len(x)
19     while alpha > eps:
20         improved = False
21         for d in rand_positive_spanning_set(alpha, n):
22             x_new = x + alpha * d; y_new = f(x_new)
23             if y_new < y:
24                 x, y, improved = x_new, y_new, True
25                 x2 = x + 3*alpha*d; y2 = f(x2)          # opportunistic probe
26                 if y2 < y: x, y = x2, y2
27                 break
```

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) I

Training or fine-tuning a large language model (LLM) with billions of parameters requires not only storing the parameters themselves but also storing the *gradient* of every parameter — and, during backpropagation, the *intermediate activations* at every layer. For a model with n parameters, this means roughly $3n$ to $5n$ floating-point numbers must be held in memory simultaneously, which can easily exceed the memory of available hardware.

MeZO (Memory-Efficient Zeroth-Order, Zhang et al. 2022) solves this problem by *estimating* the gradient using only two forward passes through the model, requiring $O(1)$ extra memory beyond the parameters themselves. The trick is to use a *pseudo-random number generator (PRNG) with a fixed seed* to generate the random direction, so we can re-generate the same direction for the parameter update without ever storing it.

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) II

The MeZO Gradient Estimator

At each step, MeZO samples a random unit direction $\mathbf{z}$ ($\mathbf{z}$, a random vector) from a standard multivariate Gaussian distribution, using a fixed PRNG seed s :

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

It then evaluates the objective at two nearby points and computes a *finite-difference gradient estimate*:

$$\hat{\mathbf{g}} = \frac{f(\mathbf{x} + \varepsilon \mathbf{z}) - f(\mathbf{x} - \varepsilon \mathbf{z})}{2\varepsilon} \mathbf{z}$$

The parameter update is then:

$$\mathbf{x} \leftarrow \mathbf{x} - \alpha \hat{\mathbf{g}}$$

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) III

Notation

- ε (epsilon) is the *smoothing radius* — how far we perturb $\mathbf{x}$ in the direction $\mathbf{z}$ to probe the function. A smaller ε gives a more accurate finite-difference estimate but is more sensitive to numerical noise; a larger ε is more robust but less accurate.
- $\mathbf{z}$ (z) is the random direction vector, the same dimension as $\mathbf{x}$. Because $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, it is isotropic — it points in a random direction in the full n -dimensional space.
- $f(\mathbf{x} + \varepsilon\mathbf{z})$ is a forward pass through the model at a slightly perturbed parameter set. No backward pass (backpropagation) is needed.
- $\hat{g}$ (g -hat) is the gradient estimate. It is a scalar times $\mathbf{z}$, so it always points along the random direction $\mathbf{z}$.
- α (alpha) is the learning rate (step size).

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) IV

How to Read This Formula

The fraction $\frac{f(\mathbf{x}+\varepsilon\mathbf{z})-f(\mathbf{x}-\varepsilon\mathbf{z})}{2\varepsilon}$ is the *directional derivative* of f along $\mathbf{z}$, approximated by finite differences. Multiplying by $\mathbf{z}$ converts this scalar directional derivative into a vector that estimates the full gradient. The key insight is that $\mathbb{E}[\hat{\mathbf{g}}] = \nabla f(\mathbf{x})$ — averaged over many random directions $\mathbf{z}$, this estimate is unbiased.

The Memory Trick: PRNG Seeds

After computing $\hat{\mathbf{g}}$, we need to perform the update $\mathbf{x} \leftarrow \mathbf{x} - \alpha\hat{\mathbf{g}}$. This requires knowing $\mathbf{z}$ again. Instead of storing $\mathbf{z}$ (which would require $O(n)$ memory), we *re-initialise* the PRNG with the same seed s and regenerate $\mathbf{z}$ from scratch. Since a PRNG always produces the same sequence from the same seed, we get the identical vector $\mathbf{z}$ without any extra storage. The update then modifies each parameter x_i by $-\alpha \cdot \hat{g}_{\text{scalar}} \cdot z_i$, computed one element at a time, requiring only $O(1)$ extra memory regardless of model size.

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) V

Worked Numerical Example (One Dimension)

Consider $f(x) = x^2$, starting at $x_0 = 3$, with $\varepsilon = 0.3$ and $\alpha = 0.1$.

The random direction in 1D is just a scalar z . Suppose $z = 1$ (drawn from the standard normal).

$$f(x_0 + \varepsilon z) = f(3 + 0.3 \times 1) = f(3.3) = 10.89$$

$$f(x_0 - \varepsilon z) = f(3 - 0.3 \times 1) = f(2.7) = 7.29$$

$$\hat{g}_{\text{scalar}} = \frac{10.89 - 7.29}{2 \times 0.3} = \frac{3.6}{0.6} = 6.0$$

The true gradient at $x = 3$ is $f'(3) = 2 \times 3 = 6$. The estimate is exact here because $f(x) = x^2$ is quadratic and the central-difference formula is exact for polynomials up to degree 2.

The update: $x_1 = 3 - 0.1 \times 6.0 \times 1 = 3 - 0.6 = 2.4$.

8.3 Memory-Efficient Zeroth-Order Optimization (MeZO) VI

Key Properties of MeZO

- **Memory usage:** $O(1)$ extra memory beyond the model parameters — just two scalar function values and a seed.
- **Unbiasedness:** $\mathbb{E}[\hat{g}] = \nabla f(\mathbf{x})$ by the spherical symmetry of the Gaussian distribution.
- **No backward pass:** Only two forward passes are needed per update, which is possible even for models that are not differentiable or where the backward pass is unavailable.
- **Best suited for fine-tuning:** MeZO works well when the model is already close to a good solution (e.g., a pretrained LLM being fine-tuned on a new task). It is not a general replacement for SGD when training from scratch on large datasets.

8.3 MeZO — Python Implementation

```
1 import numpy as np
2
3 def mezo_step(f, x, eps=1e-3, alpha=1e-3, seed=None):
4     """
5     One MeZO step. Re-uses PRNG seed to avoid storing z.
6     f      : objective (x -> scalar)
7     x      : current parameters (modified in place)
8     eps    : smoothing radius (epsilon)
9     alpha  : learning rate
10    """
11    rng = np.random.default_rng(seed)
12    z = rng.standard_normal(x.shape)          # random direction
13    # Two function evaluations (forward passes only)
14    f_plus = f(x + eps * z)
15    f_minus = f(x - eps * z)
16    grad_est = (f_plus - f_minus) / (2 * eps)  # scalar directional derivative
17    # Re-generate same z using same seed for in-place update
18    rng2 = np.random.default_rng(seed)
19    z2 = rng2.standard_normal(x.shape)
20    x -= alpha * grad_est * z2                 # O(1) extra memory
21    return x
22
23 def mezo_optimize(f, x0, n_iter=500, eps=1e-3, alpha=1e-3):
24     x = np.array(x0, dtype=float)
25     for k in range(n_iter):
26         x = mezo_step(f, x, eps=eps, alpha=alpha, seed=k)
27     return x
```

8.4 Simulated Annealing — Core Idea I

Simulated annealing (SA) is inspired by a physical process from metallurgy: when a metal is heated to a high temperature and then slowly cooled (*annealed*), its atoms settle into a low-energy crystalline structure. If the cooling is too fast, atoms get trapped in disordered, high-energy configurations — the physical analogue of a local minimum. If the cooling is slow and controlled, the system finds its global minimum energy state.

In optimisation, the “temperature” t (t) is a control parameter that governs how willing we are to accept a worse solution. At high temperature, we accept almost any move — we explore broadly. As temperature decreases, we become increasingly selective, accepting only moves that improve the objective. This smooth transition from exploration to exploitation is what allows SA to escape local minima early in the search and converge to a good solution later.

8.4 Simulated Annealing — Core Idea II

The Metropolis Acceptance Criterion

At each iteration k , we propose a new point $\mathbf{x}'$ drawn from a transition distribution $T(\mathbf{x}'|\mathbf{x}^{(k)})$, which is typically a Gaussian centred at $\mathbf{x}^{(k)}$. We then decide whether to accept or reject $\mathbf{x}'$ using the following rule:

$$\mathbf{x}^{(k+1)} \leftarrow \begin{cases} \mathbf{x}' & \text{if } f(\mathbf{x}') \leq f(\mathbf{x}^{(k)}) \quad (\text{always accept improvements}) \\ \mathbf{x}' & \text{with probability } e^{-\Delta y/t^{(k)}} \quad (\text{sometimes accept worsening}) \\ \mathbf{x}^{(k)} & \text{otherwise (stay in place)} \end{cases}$$

where $\Delta y = f(\mathbf{x}') - f(\mathbf{x}^{(k)}) > 0$ is the amount by which the proposed point is *worse*.

8.4 Simulated Annealing — Core Idea III

Notation

- $t^{(k)}$ (t superscript k) is the *temperature* at iteration k . It is a positive real number that decreases over time according to a *cooling schedule*.
- Δy (Delta y , where Δ (Delta) means “change in”) is the increase in objective value from accepting the proposed point. Since we are minimising, $\Delta y > 0$ means the proposed point is worse.
- $e^{-\Delta y / t^{(k)}}$ (e to the power of negative Delta- y over t - k) is the *acceptance probability* for a worsening move. Here $e \approx 2.718$ is Euler’s number.

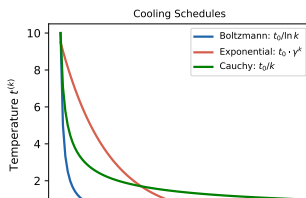
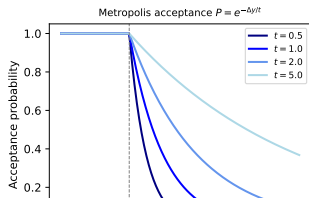
8.4 Simulated Annealing — Core Idea IV

How to Read the Acceptance Probability

The formula $e^{-\Delta y/t}$ is always between 0 and 1 (since $\Delta y > 0$ and $t > 0$), so it is a valid probability. Two things determine this probability:

- **How bad is the proposed move (Δy)?** A small Δy (slightly worse) gives a high acceptance probability; a large Δy (much worse) gives a very low acceptance probability. This is sensible — we are more likely to accept a small setback than a catastrophic one.
- **What is the temperature (t)?** At high t , $e^{-\Delta y/t} \approx 1$ and almost everything is accepted. At low t , $e^{-\Delta y/t} \approx 0$ and only improvements are accepted.

This is directly analogous to the physical process: a hot metal atom can easily overcome an energy barrier, while a cold atom is unlikely to.



Left: Acceptance probability $e^{-\Delta y/t}$ as a function of Δy (the amount the proposed point is worse) for various temperatures t . At high t , the curve is flat near 1 (everything accepted); at low t , it drops sharply to 0.

Right: Three standard cooling schedules

8.4 Simulated Annealing — Core Idea V

Standard Cooling Schedules

The cooling schedule determines how the temperature $t^{(k)}$ decreases from iteration to iteration. Three standard choices are:

Name	Formula	Convergence	Comment
Boltzmann	$t^{(k)} = t^{(1)} / \ln(k)$	Asymptotically guaranteed	Very slow
Cauchy	$t^{(k)} = t^{(1)} / k$	Guaranteed under conditions	Faster
Exponential	$t^{(k)} = t^{(1)} \cdot \gamma^k$	Not guaranteed	Most practical

Here $t^{(1)}$ (t superscript 1) is the initial temperature, $\ln(\cdot)$ ($\ln$) denotes the natural logarithm, and γ (gamma) is a constant slightly less than 1 (such as $\gamma = 0.95$) for the exponential schedule.

8.4 Simulated Annealing — Core Idea VI

Boltzmann Annealing: Slow but Theoretically Sound

The Boltzmann schedule pairs with a *Gaussian* transition distribution: we propose $\mathbf{x}' = \mathbf{x} + t^{(k)}\mathcal{N}(\mathbf{0}, \mathbf{I})$. The proposal spread shrinks with temperature, which together with the $1/\ln(k)$ cooling guarantees that the method converges to the global minimum — but only *asymptotically* (as $k \rightarrow \infty$), which in practice can require an impractically large number of iterations.

Fast Simulated Annealing (Cauchy Schedule)

The Cauchy/Lorentz transition distribution has *heavier tails* than the Gaussian: very large jumps are much more likely. This allows the algorithm to explore widely even at moderate temperatures. The schedule $t^{(k)} = t^{(1)}/k$ cools faster than Boltzmann but is still slow enough that, under certain conditions, global convergence is maintained.

8.4 Simulated Annealing — Core Idea VII

Exponential Cooling: A Numerical Table

With $t^{(1)} = 10$ and $\gamma = 0.95$ (ninety-five percent of the previous temperature), the temperatures at selected iterations are:

Iteration k	Temperature $t^{(k)} = 10 \times 0.95^k$
1	9.50
10	5.99
50	0.769
100	0.059
200	0.00035

At $t = 1$, the probability of accepting a move that is 1 unit worse is $e^{-1/1} \approx 0.368$ (about 37%), and the probability of accepting a move that is 5 units worse is $e^{-5/1} \approx 0.007$ (about 0.7%). At $t = 0.001$, even a move that is 0.01 units worse has acceptance probability $e^{-10} \approx 0.00005$ — effectively never accepted.

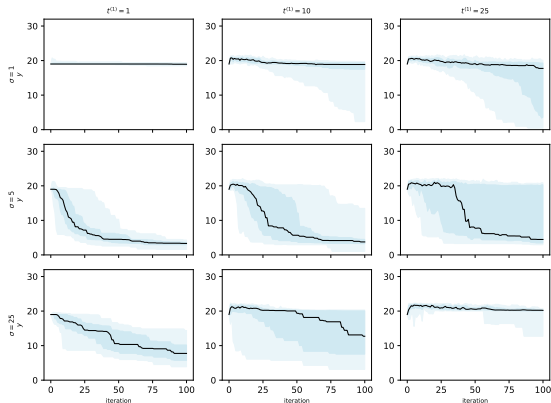
8.4 Simulated Annealing — Python Implementation

```
1 import numpy as np
2
3 def simulated_annealing(f, x0, t_init=10.0, t_schedule='boltzmann',
4                         sigma=1.0, n_iter=1000):
5     """
6     f            : objective function
7     x0           : starting point
8     t_init       : initial temperature
9     t_schedule   : 'boltzmann' | 'cauchy' | 'exponential'
10    sigma        : transition distribution std. dev.
11    """
12    x = np.array(x0, dtype=float)
13    y = f(x)
14    best_x, best_y = x.copy(), y
15    gamma = 0.95 # for exponential schedule
16
17    for k in range(1, n_iter + 1):
18        # Temperature schedule
19        if t_schedule == 'boltzmann':
20            t = t_init / np.log(k + 1)
21        elif t_schedule == 'cauchy':
22            t = t_init / k
23        else: # exponential
24            t = t_init * gamma**k
25
26        # Propose new point (Gaussian transition)
27        x_new = x + np.random.randn(len(x)) * sigma
```

8.4 SA — Effect of Temperature and Variance (Example 8.2) I

The Ackley function is a classic benchmark with many local minima arranged in a highly irregular landscape. It has a global minimum of 0 at $\mathbf{x} = \mathbf{0}$, surrounded by many local minima that can trap gradient-based methods.

SA on Ackley's Function: σ vs Temperature (Example 8.2)



What the Experiment Shows

The key variable is the standard deviation σ (sigma) of the Gaussian transition distribution — it controls how far the algorithm can jump in a single step.

- **Small** $\sigma = 1$: The algorithm can only make small moves. Local minima near the starting point will trap it, because the probability of jumping over the barrier to a better minimum is negligible.
- **Large** $\sigma = 25$: The algorithm makes very large, essentially random jumps. It explores broadly but wastes most of its function evaluations on points far from any minimum.

8.4 SA — Effect of Temperature and Variance (Example 8.2) II

Key Takeaway

For functions with widely-separated local minima, match the transition distribution spread to the typical distance between minima. The temperature schedule controls *when* to stop exploring; the spread controls *how far* you can explore in each step.

8.4 Adaptive Simulated Annealing (Algorithm 8.7) I

Standard SA uses the same step distribution for all coordinates, which is inefficient when the function varies at very different scales in different directions. Adaptive SA (Ingber, 1989) assigns each coordinate i its own step vector v_i and adjusts it so that the acceptance rate in that direction stays near 50%.

8.4 Adaptive Simulated Annealing (Algorithm 8.7) II

The Adaptive SA Procedure

- 1 Run a *cycle* of n_s random moves, varying one coordinate at a time.
- 2 Count A_i = number of accepted moves in coordinate direction i .
- 3 Adjust the step size v_i (v sub i) for each coordinate:

$$v_i \leftarrow \begin{cases} v_i \left(1 + c_i \frac{A_i/n_s - 0.6}{0.4} \right) & \text{if } A_i > 0.6 n_s \quad (\text{accept rate too high, increase step}) \\ v_i \left(1 + c_i \frac{0.4 - A_i/n_s}{0.4} \right)^{-1} & \text{if } A_i < 0.4 n_s \quad (\text{accept rate too low, decrease step}) \\ v_i & \text{otherwise} \quad (\text{accept rate in target range}) \end{cases}$$

- 4 Repeat for n_t temperature-reduction cycles, then lower the temperature.
- 5 When no adjustment is needed for n_ϵ consecutive cycles, add the current point to the list of candidate optima.

8.4 Adaptive Simulated Annealing (Algorithm 8.7) III

Notation

- v_i (v sub i) is the step size for coordinate i . Different coordinates can have very different step sizes, which is the key advantage over standard SA.
- A_i (A sub i) is the number of accepted moves out of n_s (n sub s) attempts in direction i .
- A_i/n_s is the empirical acceptance rate, which we target to be between 0.4 and 0.6 (40% to 60%).
- c_i (c sub i) is a tuning constant (typically $c_i = 2$) that controls how aggressively the step size is adjusted.

Why Target 50% Acceptance?

A 50% acceptance rate balances exploration and exploitation: if almost all moves are accepted, the step is too small and the algorithm is not exploiting the objective; if almost no moves are accepted, the step is too large and the algorithm is wasting evaluations on proposals that will always be rejected.

8.4 Adaptive Simulated Annealing (Algorithm 8.7) IV



After five iterations on the domain function, appendix B.3. The right plot shows the interval objective function values versus their radii, which is useful for identifying intervals to split with further evaluations.

8.5 Cross-Entropy Method I

The Cross-Entropy Method (CEM) takes a fundamentally different approach to optimisation: instead of maintaining a single current point, it maintains a *probability distribution* over the design space. The distribution is iteratively updated to concentrate probability mass near the best solutions found so far. Ultimately, the distribution collapses onto the global minimum.

The name “cross-entropy” comes from information theory: the algorithm minimises the cross-entropy between the current proposal distribution and an ideal target distribution concentrated at the global minimum. In practice, we achieve this by fitting the distribution to the *elite samples* — the best-performing subset of samples drawn from the current distribution.

8.5 Cross-Entropy Method II

The CEM Algorithm

- 1 **Sample:** Draw m (m) candidate solutions $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ from the current proposal distribution $P_{\theta}(\mathbf{x})$, where θ (theta) are the distribution parameters (e.g., mean and covariance for a Gaussian).
- 2 **Evaluate and rank:** Compute $f(\mathbf{x}^{(i)})$ for each sample and rank the samples from best (lowest f) to worst. Select the top m_{elite} (m subscript “elite”) samples as the *elite set* $\mathcal{E}$.
- 3 **Refit:** Update P_{θ} by fitting it to the elite set using maximum likelihood estimation (MLE). For a Gaussian distribution, MLE gives the sample mean and covariance of the elite set.
- 4 **Repeat** until the distribution is sufficiently concentrated (e.g., standard deviation below a threshold).

8.5 Cross-Entropy Method III

MLE Update Formulas for a Gaussian Proposal $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$

After selecting the m_e (m sub e) elite samples $\{\mathbf{x}^{(i)}\}_{i=1}^{m_e}$, we update the distribution parameters by:

$$\boldsymbol{\mu}^{(k+1)} = \frac{1}{m_e} \sum_{i=1}^{m_e} \mathbf{x}^{(i)}$$

$$\boldsymbol{\Sigma}^{(k+1)} = \frac{1}{m_e} \sum_{i=1}^{m_e} \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)} \right) \left(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)} \right)^\top$$

8.5 Cross-Entropy Method IV

Notation for the MLE Updates

- $\boldsymbol{\mu}^{(k+1)}$ (bold mu, superscript $k + 1$) is the new mean vector of the proposal distribution. It is the arithmetic average (centroid) of the elite samples.
- $\boldsymbol{\Sigma}^{(k+1)}$ (bold Sigma, superscript $k + 1$) is the new covariance matrix. Each term $(\mathbf{x}^{(i)} - \boldsymbol{\mu})(\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top$ is an outer product (a matrix formed from a column vector times its own transpose), and we average these over all elite samples.
- $\boldsymbol{\Sigma}$ (Sigma) is an $n \times n$ symmetric positive-definite matrix that encodes both the variances of each coordinate and the correlations between coordinates.

How to Read This Update

The mean $\boldsymbol{\mu}$ moves toward the centroid of the elite samples, and the covariance $\boldsymbol{\Sigma}$ shrinks to reflect the spread of those samples. Over iterations, both the mean and the spread decrease: the distribution concentrates around the region of best performance. This is analogous to “zooming in” on the most promising part of the design space.

8.5 Cross-Entropy Method V

26 CHAPTER 7. DIRECT METHODS

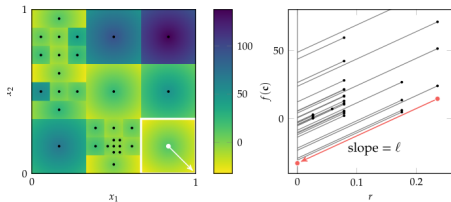


Figure 7.14. The left plot shows the Lipschitz bounds constructed for the RECT intervals using the Lipschitz constant $\ell = 200$, but now it highlights the interval containing lowest value. The right plot shows how the lowest lower bound given Lipschitz constant is the one with the lowest x -intercept in right-hand plot.

CEM applied to a 1D function (shown as a blue curve). The Gaussian proposal distribution (shaded region) starts broad and gradually narrows onto the global minimum over successive iterations.

8.5 Cross-Entropy Method VI

Worked Example: CEM on the Branin Function (Example 8.3)

The Branin function is a 2D test function with three global minima all achieving $f^* \approx 0.398$. Starting from $\mu_0 = (0.5, 5.0)$ with $\Sigma_0 = \mathbf{I}$ (the identity matrix), using $m = 40$ samples per iteration and $m_{\text{elite}} = 10$ elite samples:

Iteration	μ_1	μ_2	$f(\mu)$
0	0.50	5.00	51.8
2	-0.12	4.21	12.3
4	-2.1	12.1	3.4
6	-3.1	12.3	0.9
8	-3.14	12.27	0.40
10	-3.14	12.27	0.398

In just 10 iterations (400 total function evaluations), the algorithm converges to the global minimum from a starting point with $f = 51.8$ — a reduction of more than 99%.

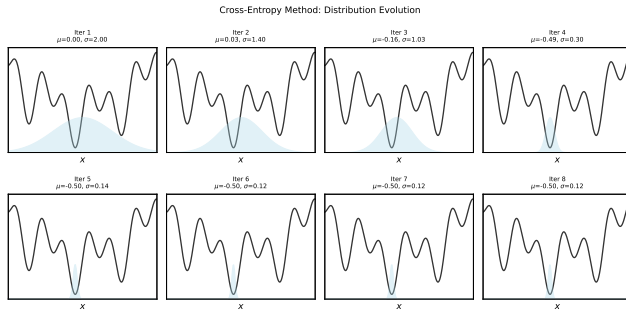
8.5 Cross-Entropy Method VII

Choosing the Elite Fraction

The ratio m_{elite}/m is a critical hyperparameter:

- If $m_{\text{elite}}/m \rightarrow 1$: the refitted distribution closely matches the current proposal, meaning the update is very small and convergence is very slow.
- If $m_{\text{elite}}/m \rightarrow 0$: only a tiny number of samples drive the update, making it highly noisy and potentially unstable.
- Typical choice: $m_{\text{elite}} \approx m/10$. This provides a stable but responsive update.

8.5 Cross-Entropy Method VIII



CEM applied to a 1D multi-modal function. The Gaussian proposal (blue shading) narrows onto the global minimum over iterations, ignoring the local minima.

8.5 Cross-Entropy Method — Python Implementation

```
1 import numpy as np
2
3 def cross_entropy_method(f, mu0, Sigma0, k_max=50, m=100, m_elite=10):
4     """
5     f          : objective to MINIMISE  $R^n \rightarrow R$ 
6     mu0        : initial mean (n,)
7     Sigma0     : initial covariance (n,n)
8     m          : total samples per iteration
9     m_elite    : number of elite samples used for refitting
10    Returns    : estimated minimiser (final mean)
11    """
12    mu = np.array(mu0, dtype=float)
13    Sigma = np.array(Sigma0, dtype=float)
14
15    for k in range(k_max):
16        # Sample from proposal distribution
17        samples = np.random.multivariate_normal(mu, Sigma, m) # (m, n)
18        values = np.array([f(x) for x in samples])
19
20        # Select elite samples (lowest f values)
21        elite_idx = np.argsort(values)[:m_elite]
22        elite = samples[elite_idx]
23
24        # Refit distribution (MLE for multivariate Gaussian)
25        mu = elite.mean(axis=0)
26        diff = elite - mu
27        Sigma = (diff.T @ diff) / m_elite + np.eye(len(mu)) * 1e-6
```

8.6 Proposal Distribution Descent (PDD) I

The Cross-Entropy Method updates the proposal distribution by fitting it to elite samples — a heuristic procedure. Proposal Distribution Descent (PDD) takes a more principled approach: it directly differentiates the *expected objective* with respect to the distribution parameters θ (bold theta), and performs gradient descent on those parameters.

The goal is to minimise the expected value of f under the proposal distribution:

$$J(\theta) = \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x})]$$

Notation for the Expected Objective

- $J(\theta)$ (J of theta) is the objective we want to minimise — the average of $f(\mathbf{x})$ when $\mathbf{x}$ is drawn randomly from our proposal distribution p_θ .
- θ (bold theta) are the parameters of the proposal distribution. For a Gaussian, $\theta = (\mu, \nu)$ where μ (mu) is the mean and ν (nu) is the variance.
- $\mathbb{E}_{\mathbf{x} \sim p_\theta} [\cdot]$ (E subscript x tilde p-theta) means “expected value when $\mathbf{x}$ is sampled from p_θ ”.

8.6 Proposal Distribution Descent (PDD) II

Score Function Gradient (the REINFORCE Trick)

Computing $\nabla_{\theta} J$ is non-trivial because the distribution p_{θ} itself depends on θ . The score function estimator (also known as the REINFORCE trick) gives:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\theta}} [f(\mathbf{x}) \nabla_{\theta} \log p_{\theta}(\mathbf{x})]$$

The term $\nabla_{\theta} \log p_{\theta}(\mathbf{x})$ (nabla-theta of log p-theta of x) is called the *score function*. It measures how much the log-probability of observing $\mathbf{x}$ changes when we tweak the distribution parameters θ .

This expectation can be estimated by a Monte Carlo average over m samples:

$$\nabla_{\theta} J \approx \frac{1}{m} \sum_{i=1}^m f(\mathbf{x}^{(i)}) \nabla_{\theta} \log p_{\theta}(\mathbf{x}^{(i)})$$

where each $\mathbf{x}^{(i)} \sim p_{\theta}$. This is an unbiased estimator of the true gradient.

8.6 Proposal Distribution Descent (PDD) III

Explicit Score Functions for a 1D Gaussian $\mathcal{N}(\mu, \nu)$

For a one-dimensional Gaussian distribution with mean μ (mu) and variance ν (nu), the score functions with respect to each parameter are:

$$\nabla_{\mu} \log \mathcal{N}(x; \mu, \nu) = \frac{x - \mu}{\nu} \quad \text{and} \quad \nabla_{\nu} \log \mathcal{N}(x; \mu, \nu) = \frac{(x - \mu)^2 - \nu}{2\nu^2}$$

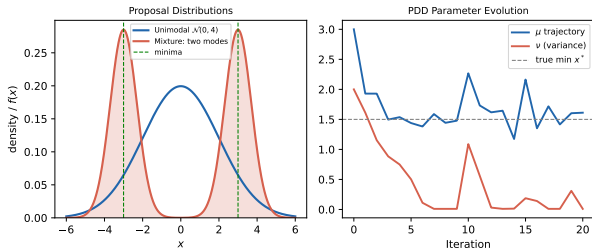
These tell us: if we see a sample x that is larger than μ , the score with respect to μ is positive, meaning increasing μ increases the probability of observing x . Similarly, a sample with a squared deviation $(x - \mu)^2$ larger than ν indicates the distribution is too narrow.

8.6 Proposal Distribution Descent (PDD) IV

Important Numerical Issue: Variance Can Go Negative

When the mean μ is near the optimum x^* , the gradient with respect to ν (nu, the variance) is *positive*. Gradient descent on ν therefore *decreases* ν toward zero. As $\nu \rightarrow 0$, the term $1/(2\nu^2)$ in the score function grows without bound, making the gradient estimate very large and noisy. With any fixed step size α , the update $\nu \leftarrow \nu - \alpha \nabla_{\nu} J$ will eventually drive ν to a negative value — which is invalid since a variance must be positive.

The fix is to reparameterise: let $\nu = e^{\eta}$ (e to the power eta), so that ν is always positive regardless of the value of η (eta). We then perform gradient descent on η instead.



Left: A unimodal Gaussian proposal concentrates on a single mode, while a mixture proposal can capture multiple modes simultaneously. **Right:** PDD parameter evolution (μ and ν) on a noisy multi-modal objective over iterations.

8.6 PDD — Python Implementation

```
1 import numpy as np
2
3 def proposal_distribution_descent(f, mu0, nu0, alpha=0.1, m=50, n_iter=200):
4     """
5     Minimise  $E_{x \sim N(\mu, \nu)}[f(x)]$  via score-function gradient.
6     f      : objective  $R \rightarrow R$ 
7     mu0    : initial mean
8     nu0    : initial variance ( $>0$ )
9     """
10    mu, log_nu = float(mu0), np.log(float(nu0)) #  $\nu = \exp(\log_\nu) > 0$ 
11
12    for _ in range(n_iter):
13        nu = np.exp(log_nu)
14        xs = np.random.normal(mu, np.sqrt(nu), m)
15        fs = np.array([f(x) for x in xs])
16
17        # Score-function gradients
18        score_mu = (xs - mu) / nu
19        score_nu = ((xs - mu)**2 - nu) / (2 * nu**2)
20
21        g_mu = np.mean(fs * score_mu)
22        g_nu = np.mean(fs * score_nu)
23
24        # Update (reparameterise  $\nu = \exp(\log_\nu)$  to keep  $\nu > 0$ )
25        g_log_nu = g_nu * nu # chain rule
26        mu -= alpha * g_mu
27        log_nu -= alpha * g_log_nu
```


8.7 Information-Geometric Optimization (IGO) I

PDD performs gradient descent in the space of distribution parameters θ . However, this ordinary gradient is sensitive to how we parameterise the distribution: a different but equivalent parameterisation (e.g., using the precision $1/\nu$ instead of the variance ν) gives a different gradient and a different optimisation trajectory. This lack of *invariance* is a practical problem: the algorithm's behaviour depends on an arbitrary modelling choice.

Information-Geometric Optimization (IGO) fixes this by using the *natural gradient* instead of the ordinary gradient. The natural gradient accounts for the intrinsic geometry of the space of probability distributions, making the update invariant to reparameterisation.

8.7 Information-Geometric Optimization (IGO) II

The Fisher Information Matrix

The Fisher information matrix $\mathbf{F}(\boldsymbol{\theta})$ (bold F of theta) measures how much information a single observation from $p_{\boldsymbol{\theta}}$ carries about the parameters $\boldsymbol{\theta}$:

$$\mathbf{F}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x} \sim p_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{x}) (\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{x}))^{\top} \right]$$

This is a square matrix of size (number of parameters) $\times$ (number of parameters). It acts as a local “metric tensor” on the space of distributions — it measures how different two nearby distributions are in terms of the information they carry.

8.7 Information-Geometric Optimization (IGO) III

The Natural Gradient Update

The natural gradient $\tilde{\nabla}_{\theta} J$ is obtained by pre-multiplying the ordinary gradient by the inverse Fisher information matrix:

$$\tilde{\nabla}_{\theta} J = \mathbf{F}^{-1}(\theta) \nabla_{\theta} J(\theta)$$

The IGO update is then:

$$\theta^{(k+1)} \leftarrow \theta^{(k)} - \alpha \tilde{\nabla}_{\theta} J(\theta^{(k)})$$

Pre-multiplying by $\mathbf{F}^{-1}$ (F inverse) “stretches” or “squashes” the gradient vector to correct for the curvature of the distribution manifold, just as one would correct for the curvature of a physical surface when computing shortest paths.

8.7 Information-Geometric Optimization (IGO) IV

How to Read the Natural Gradient

Think of the parameter space as a curved surface (a Riemannian manifold). In flat Euclidean space, the steepest descent direction is simply $-\nabla J$. On a curved surface, the steepest descent must account for the surface's curvature — otherwise a step that looks large in parameter space may correspond to a tiny change in the actual distribution. The Fisher matrix $\mathbf{F}$ encodes this curvature, and $\mathbf{F}^{-1}\nabla J$ is the correctly scaled steepest descent direction on the curved surface.

Natural Gradient for a Multivariate Gaussian $\mathcal{N}(\mu, \Sigma)$

For a Gaussian proposal, the natural gradient updates simplify to remarkably clean formulas (equations 8.25–8.26 in the textbook):

$$\begin{aligned}\tilde{\nabla}_{\mu} J &= \mathbb{E}[f(\mathbf{x})(\mathbf{x} - \mu)] \\ \tilde{\nabla}_{\Sigma} J &= \frac{1}{2} \mathbb{E}[f(\mathbf{x})((\mathbf{x} - \mu)(\mathbf{x} - \mu)^{\top} - \Sigma)]\end{aligned}$$

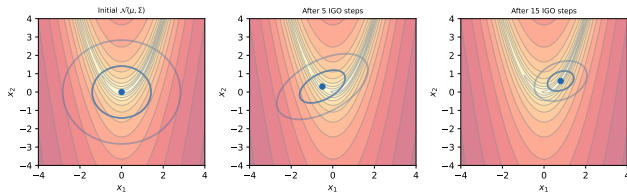
These are estimated by Monte Carlo averaging over samples.

8.7 Information-Geometric Optimization (IGO) V

Why IGO Uses Fitness Rankings, Not Raw Values

A key property of IGO is that the update is invariant to any *order-preserving transformation* of f : replacing $f(\mathbf{x})$ with $g(f(\mathbf{x}))$ for any increasing function g produces the same distribution trajectory. In practice, this is implemented by replacing raw f values with *rank-based weights*: the best sample gets weight w_1 , the second-best gets w_2 , etc. This makes IGO robust to outliers and eliminates the need to tune the absolute scale of f .

Information-Geometric Optimization (IGO) on Banana Function



IGO distributions converging on the Banana (Rosenbrock) function, a notoriously difficult test problem with a curved, narrow valley. The ellipses show the 1-sigma contours of the Gaussian proposal at successive iterations. Fisher information removes scale dependence, allowing the ellipse to align with the valley.

8.7 IGO — Python Implementation (Algorithm 8.10)

```
1 import numpy as np
2
3 def information_geometric_optimization(f, mu0, Sigma0, k_max=100, alpha=1.0):
4     """
5     Algorithm 8.10: IGO with multivariate normal.
6     Sample size  $m = 4 + \text{floor}(3 \cdot \log(n))$ .
7     Elite fraction: half the samples (ranked weighting).
8     """
9     mu = np.array(mu0, dtype=float)
10    Sigma = np.array(Sigma0, dtype=float)
11    n = len(mu)
12
13    m = 4 + int(3 * np.log(n))          # population size
14    m_elite = m // 2                    # number of elite samples
15
16    # Log-based weights (eq. 8.29): rank 1 = best
17    ws = np.log(m_elite + 1) - np.log(np.arange(1, m_elite + 1))
18    ws /= ws.sum()                     # normalise to sum to 1
19
20    for _ in range(k_max):
21        # Sample and evaluate
22        xs = np.random.multivariate_normal(mu, Sigma, m)
23        ys = np.array([f(x) for x in xs])
24        order = np.argsort(ys)         # best to worst
25
26        # Weighted natural gradient updates
27        delta_mu = np.zeros(n)
```

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) I

CMA-ES (Hansen & Ostermeier, 2001) extends IGO by making a critical distinction: it separates the *overall scale* of the search (controlled by a scalar step size σ (sigma)) from the *shape and orientation* of the search (controlled by a shape matrix **C** (bold C)).

This separation allows CMA-ES to independently adapt how large to search (expanding when progress is fast, contracting when it stalls) and which directions to search preferentially (rotating and stretching the search ellipsoid to align with the problem's natural axes).

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) II

The CMA-ES Sampling Distribution

At each iteration, candidate solutions are drawn from:

$$\mathbf{x}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{C})$$

where:

- $\boldsymbol{\mu}$ (bold mu) is the current mean of the distribution — the algorithm's best estimate of the location of the minimum.
- σ (sigma) is the global step size — a positive scalar controlling the overall scale of the search. Adapting σ is analogous to adapting the learning rate in SGD.
- $\mathbf{C}$ (bold C) is the shape matrix (a covariance matrix normalised so that its determinant is 1). It encodes the preferred search directions. When $\mathbf{C} = \mathbf{I}$ (the identity matrix), search is isotropic (equal in all directions). When $\mathbf{C}$ is elongated, search is preferentially in the elongated direction.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) III

The Mean Update (Same as IGO)

$$\boldsymbol{\mu}' = \boldsymbol{\mu} + \alpha_{\mu} \sum_{i=1}^m w_i (\mathbf{x}^{[i]} - \boldsymbol{\mu})$$

Here $[i]$ (brackets around i) means the i -th best sample (rank 1 = best), w_i (w sub i) are positive weights that sum to 1, and α_{μ} (alpha sub μ) is a step size for the mean update (typically 1). This is a weighted average update: the mean moves toward the weighted centroid of the elite samples.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) IV

Evolution Paths: Accumulating Direction Information

CMA-ES introduces two “evolution paths” — running averages of the steps taken by the mean over recent iterations. These paths accumulate information about the direction and speed of progress. The **sigma-path** $\mathbf{p}_\sigma$ (p sub sigma) tracks the cumulated step direction for step-size control:

$$\mathbf{p}_\sigma^{(k+1)} = (1 - \alpha_\sigma)\mathbf{p}_\sigma^{(k)} + \sqrt{\alpha_\sigma(2 - \alpha_\sigma)m_{\text{eff}}} \frac{\boldsymbol{\mu}' - \boldsymbol{\mu}}{\sigma}$$

The **C-path** $\mathbf{p}_c$ (p sub c) tracks the cumulated step direction for covariance adaptation:

$$\mathbf{p}_c^{(k+1)} = (1 - \alpha_c)\mathbf{p}_c^{(k)} + \sqrt{\alpha_c(2 - \alpha_c)m_{\text{eff}}} \frac{\boldsymbol{\mu}' - \boldsymbol{\mu}}{\sigma}$$

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) V

Notation for the Evolution Paths

- α_σ (alpha sub sigma) and α_c (alpha sub c) are decay constants in $(0, 1)$ controlling how quickly old information is forgotten. Typical values are around $1/\sqrt{n}$.
- $(1 - \alpha_\sigma)\mathbf{p}_\sigma^{(k)}$ is the “faded” old path — old information is down-weighted exponentially.
- The second term adds the current normalised step $(\boldsymbol{\mu}' - \boldsymbol{\mu})/\sigma$, scaled by $\sqrt{\alpha_\sigma(2 - \alpha_\sigma)m_{\text{eff}}}$ to keep the path length statistically calibrated.
- m_{eff} (m sub eff) is the effective population size: $m_{\text{eff}} = 1/\sum_i w_i^2$. It measures how many samples are contributing effectively to the update. A high m_{eff} means the weights are spread evenly; a low m_{eff} means one or two samples dominate.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) VI

Why Evolution Paths Are Powerful

Without evolution paths, the covariance update would only use information from the current iteration's samples — a very noisy signal. By accumulating the direction of movement across many iterations, the evolution path provides a much cleaner signal. If the algorithm consistently moves in the same direction over multiple iterations, the evolution path becomes long and the algorithm learns to preferentially search in that direction.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) VII

Step-Size Control

The global step size σ is updated based on the length of the sigma-path:

$$\sigma^{(k+1)} = \sigma^{(k)} \exp\left(\frac{\alpha_\sigma}{\beta_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}[\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|]} - 1\right)\right)$$

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) VIII

Notation for Step-Size Control

- $\|\mathbf{p}_\sigma\|$ (the norm, or length, of the sigma-path) is compared to $\mathbb{E}[\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|]$, the expected length of a standard Gaussian random vector. This expected length is $\sqrt{2} \Gamma((n+1)/2) / \Gamma(n/2)$ where Γ (Gamma) is the Gamma function (a generalisation of the factorial).
- If $\|\mathbf{p}_\sigma\|$ is *larger* than expected, the algorithm has been consistently moving in the same direction — a sign that the step size should be *increased* to make faster progress.
- If $\|\mathbf{p}_\sigma\|$ is *smaller* than expected, the algorithm has been oscillating, changing direction frequently — a sign that the step size should be *decreased* to improve accuracy.
- β_σ (beta sub sigma) is a damping coefficient (typically 1.0) that prevents the step size from changing too rapidly.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) IX

Covariance Matrix Update

The shape matrix $\mathbf{C}$ is updated using a combination of a rank-1 update (from the C-path) and a rank- m update (from the current elite samples):

$$\mathbf{C}^{(k+1)} = (1 - \alpha_1 - \alpha_e)\mathbf{C}^{(k)} + \alpha_1\mathbf{p}_c\mathbf{p}_c^\top + \frac{\alpha_e}{\sigma^2} \sum_{i=1}^m w_i \mathbf{x}^{[i]}(\mathbf{x}^{[i]})^\top$$

- The **rank-1 update** ($\alpha_1\mathbf{p}_c\mathbf{p}_c^\top$) stretches the covariance in the direction of the evolution path, exploiting correlated steps accumulated over time. Here α_1 (alpha sub 1) controls the learning rate of this term.
- The **rank- m update** (the sum over elite samples) uses the current generation's best samples to update the covariance, exploiting correlations within a single iteration. Here α_e (alpha sub e) is the corresponding learning rate.

8.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES) X

Default Hyperparameters for an n -Dimensional Problem

Parameter	Value
Population size m	$4 + \lfloor 3 \ln n \rfloor$
Elite size m_e	$m/2$ (rounded down)
Rank-1 learning rate α_1	$2/n^2$
Rank- m learning rate α_e	m_e/n^2
C-path decay α_c	$1/\sqrt{n}$
σ -path decay α_σ	$1/\sqrt{n}$
Damping β_σ	1.0

These defaults are carefully derived from the statistical properties of the algorithm and work well across a wide range of problems without tuning.

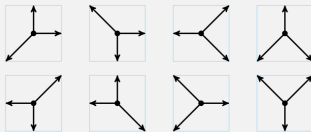
8.8 CMA-ES — Python Implementation (Algorithm 8.11)

```
1 import numpy as np
2 from scipy.special import gamma as gamma_fn
3
4 def cmaes(f, mu0, sigma0=1.0, C0=None, k_max=500):
5     n = len(mu0)
6     mu = np.array(mu0, dtype=float)
7     sig = float(sigma0)
8     C = np.eye(n) if C0 is None else np.array(C0, dtype=float)
9
10    m = 4 + int(3 * np.log(n))
11    m_elite = m // 2
12    a1 = 2 / n**2;          ae = m_elite / n**2
13    ac = 1 / np.sqrt(n);    ao = 1 / np.sqrt(n)
14    amu = 1 - a1 - ae;      bs = 1.0
15    ws = np.log(m_elite+1) - np.log(np.arange(1, m_elite+1))
16    ws /= ws.sum()
17    m_eff = 1 / np.sum(ws**2)
18    exp_len = (np.sqrt(2) * gamma_fn((n+1)/2) / gamma_fn(n/2))
19    p_sig = np.zeros(n);    p_c = np.zeros(n)
20
21    for _ in range(k_max):
22        xs = np.random.multivariate_normal(mu, sig**2 * C, m)
23        ys = np.array([f(x) for x in xs])
24        idx = np.argsort(ys)          # best first
25
26        mu_new = mu + amu * sum(ws[i]*(xs[idx[i]]-mu) for i in range(m_elite))
27        delta = (mu_new - mu) / sig
```

8.8 CMA-ES — Flower Function Example I

8.2. MESH ADAPTIVE DIRECT SEARCH 137

Consider positive spanning sets constructed from the nonzero directions $d_1, d_2 \in \{-1, 0, 1\}$. There are 8 positive spanning sets with 3 elements that can be constructed from these directions:



Example 8.1. Positive spanning sets for $\mathbb{R}^2$. Note that the lower triangular generation strategy can only generate the first two columns of spanning sets.

The rows and columns of $\mathbf{L}$ are then randomly permuted to obtain a matrix $\mathbf{D}$ whose columns correspond to n directions that linearly span $\mathbb{R}^n$. Each of these directions is inscribed inside a hypercube of side length $2/\sqrt{a^{(k)}}$.⁷

Two common methods for obtaining a positive spanning set from the linearly spanning set are to add one additional direction $\mathbf{d}^{(n+1)} = -\sum_{i=1}^n \mathbf{d}^{(i)}$ and to add n additional directions $\mathbf{d}^{(n+j)} = -\mathbf{d}^{(j)}$ for j in $1:n$. We use the first method in algorithm 8.2.

The step size a starts at 1, is always a power of 4, and never exceeds 1. Using a power of 4 causes the maximum possible componentwise step size taken in each iteration to be scaled by a factor of 2, as the maximum componentwise step size $a/\sqrt{a}$ has length $4^m/\sqrt{4^m} = 2^m$ for integer $m < 1$. The step size is updated according to:

$$a^{(k+1)} \leftarrow \begin{cases} a^{(k)}/4 & \text{if no improvement was found in this iteration} \\ \min(1, 4a^{(k)}) & \text{otherwise} \end{cases} \quad (8.4)$$

Mesh adaptive direct search is opportunistic and does not support dynamic ordering⁸ because, after a successful iteration, the new random pattern may not contain the previous successful direction. The algorithm queries a new design point along the accepted descent direction. If $f(\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + a\mathbf{d}) < f(\mathbf{x}^{(k-1)})$,

⁷ For each direction, $\|\mathbf{d}^{(i)}\|_\infty = 1/\sqrt{a^{(k)}}$.

⁸ See section 7.4.

CMA-ES applied to the flower function (a challenging 2D test function with rotational structure, defined in Appendix B.4 of the textbook). Each row shows the distribution's mean and covariance ellipse (the $1\text{-}\sigma$ contour of $\mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{C})$) overlaid on the objective's contour plot. Columns correspond to iterations 0, 5, 15, and 40.

8.8 CMA-ES — Flower Function Example II

What the Flower Example Shows

- **Iteration 0:** The distribution starts as a large circle (isotropic, $\mathbf{C} = \mathbf{I}$). It covers a broad region of the design space and has no preferred direction.
- **Iteration 5:** The ellipse has begun to rotate and elongate, aligning with the principal curvature direction of the objective function. The algorithm has learned from the first few generations that progress is easier in one direction than the other.
- **Iteration 15:** The covariance has shrunk substantially. The distribution is concentrated in a narrow elliptical region near the minimum, tightly tracking the curved valley of the flower function.
- **Iteration 40:** The distribution has converged: the ellipse is very small, centred near the global minimum, and oriented to match the local geometry of the objective.

8.8 CMA-ES — Flower Function Example III

Invariance Properties of CMA-ES

CMA-ES is invariant to three important classes of transformations, which make it robust in practice:

- 1 **Order-preserving transformations of f :** Replacing $f(\mathbf{x})$ with $g(f(\mathbf{x}))$ for any strictly increasing function g (e.g., taking the square root or the logarithm of f) does not change the algorithm's trajectory, because CMA-ES uses rank-based weights rather than raw function values.
- 2 **Rotation of the search space:** Rotating all coordinates by any orthogonal matrix does not affect performance, because $\mathbf{C}$ can adapt to any orientation.
- 3 **Scaling of individual coordinates:** Multiplying any coordinate by a constant does not affect performance (up to the time needed for $\mathbf{C}$ and σ to adapt).

These properties make CMA-ES particularly effective on ill-conditioned problems, where the objective varies at very different rates in different directions.

8.8 CMA-ES — Flower Function Example IV

Computational Complexity

- **Per iteration:** $O(n^2)$ for the covariance update (computing outer products and matrix sums).
- **Eigendecomposition:** Required every $O(n)$ iterations to sample from $\mathcal{N}(\mu, \sigma^2 \mathbf{C})$ efficiently; costs $O(n^3)$ but is amortised to $O(n^2)$ per iteration.
- **Recommended problem size:** n up to a few hundred. For $n > 1000$, specialised variants (e.g., sep-CMA-ES, which restricts $\mathbf{C}$ to be diagonal) are used.

Comparing Stochastic Methods I

Having studied eight different stochastic optimisation methods, it is useful to compare them across the dimensions most relevant to practical selection: the order of information used (zeroth vs. first), memory requirements, and whether global convergence is guaranteed.

Summary Comparison Table

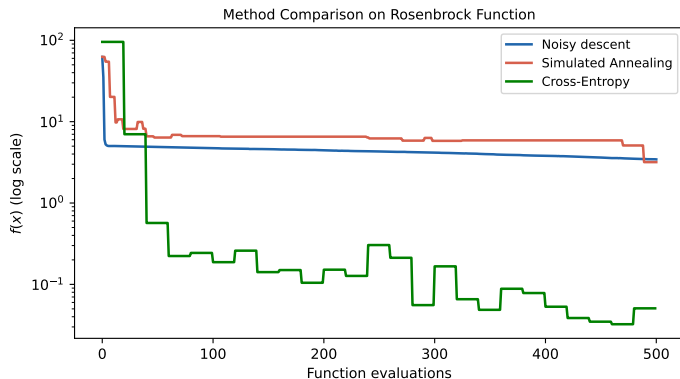
Method	Order	Memory	Global convergence?	Best suited for
Noisy Descent	1st	$O(n)$	No	Differentiable, large n , mini-batch
MADS	0th	$O(n^2)$	Asymptotic	Black-box, small n , noisy f
MeZO	0th	$O(1)$	No	Huge n (LLMs), memory-limited
Sim. Annealing	0th	$O(1)$	Asymptotic	Discrete/combinatorial, small n
Cross-Entropy	0th	$O(n^2)$	Yes (empirically)	Multi-modal, $n < 100$
PDD	0th	$O(n^2)$	No	Differentiable proposal, $n < 50$
IGO	0th	$O(n^2)$	Yes (invariant)	$n = 10$ – 100 , ill-conditioned
CMA-ES	0th	$O(n^2)$	Yes (invariant)	$n = 10$ – 100 , state of the art

Comparing Stochastic Methods II

Practical Decision Guide

- **Gradient is available and the model is large ($n > 10,000$):** Use noisy descent (SGD) with mini-batches for standard neural networks. Use MeZO if memory is the bottleneck (e.g., fine-tuning LLMs on a single GPU).
- **Black-box objective, small to medium n ($n < 20$):** MADS is a solid first choice — no gradient needed, provably convergent, and robust to noise. Simulated annealing is competitive when the problem has a combinatorial or discrete structure.
- **Many local minima, $n < 100$:** CMA-ES is generally the state-of-the-art choice. The Cross-Entropy Method is simpler to implement and competitive for moderate n .
- **Require theoretical invariance guarantees:** IGO or CMA-ES, which are invariant to reparameterisation and order-preserving transforms of f .
- **Very large n ($n > 10^6$), zeroth-order required:** MeZO, which requires only $O(1)$ extra memory regardless of model size.

Comparing Stochastic Methods III



Noisy descent, simulated annealing, and the cross-entropy method applied to the Rosenbrock function starting from $(-1.5, 1.5)$. All three reach the vicinity of the minimum at $(1, 1)$, but CEM converges most directly while SA takes a more wandering path.

8.9 Chapter Summary I

This chapter introduced stochastic optimisation methods — algorithms that deliberately incorporate randomness to escape local minima, handle noisy objectives, and explore large design spaces. Here is a concise summary of each method covered.

Noisy Descent (Section 8.1)

Noisy descent adds Gaussian random noise to the gradient at each step. The key requirement is that the noise be *unbiased* (zero mean), so that the expected update equals the true gradient direction. The Robbins–Monro step-size conditions — the step sizes must sum to infinity but their squares must sum to a finite value — guarantee convergence. A classic schedule satisfying these conditions is $\alpha^{(k)} = \alpha^{(1)}/k$.

8.9 Chapter Summary II

Mesh Adaptive Direct Search / MADS (Section 8.2)

MADS is a pattern-search method requiring only function values. It generates random positive spanning sets of search directions at each iteration using a lower-triangular random matrix, probes the objective in each direction, and updates the step size by a factor of 4 (expanding on success, contracting on failure). Because the step size is always a power of $\frac{1}{4}$, the resulting mesh becomes arbitrarily fine, guaranteeing convergence.

MeZO (Section 8.3)

MeZO estimates the gradient using only two forward function evaluations and a re-seeded PRNG. No backward pass is needed. By re-generating the random direction $\mathbf{z}$ from the same seed for the parameter update, the method requires $O(1)$ extra memory regardless of the number of parameters — making it practical for fine-tuning models with billions of parameters.

8.9 Chapter Summary III

Simulated Annealing (Section 8.4)

Simulated annealing controls exploration via a “temperature” parameter t that decreases over time. Proposed moves that *worsen* the objective are accepted with probability $e^{-\Delta y/t}$ — small for large worsening or low temperature, but substantial for small worsening at high temperature. The Boltzmann cooling schedule $t^{(k)} \propto 1/\ln(k)$ guarantees asymptotic convergence to the global minimum; the faster exponential schedule is more practical. Adaptive SA (Section 8.4) additionally adjusts the step size per coordinate to maintain a 50% acceptance rate.

Cross-Entropy Method (Section 8.5)

CEM maintains a Gaussian proposal distribution over the design space. At each iteration, it samples candidates, selects the best fraction as the elite set, and refits the Gaussian to those elite samples using maximum likelihood estimation (MLE gives the sample mean and covariance). The distribution progressively narrows onto the region of best performance. The elite fraction (typically $m_{\text{elite}} \approx m/10$) is the most critical hyperparameter.

8.9 Chapter Summary IV

Proposal Distribution Descent (Section 8.6)

PDD directly differentiates the expected objective $J(\theta) = \mathbb{E}_{x \sim p_\theta} [f(x)]$ with respect to the distribution parameters θ using the score function (REINFORCE) estimator. Unlike CEM, it does not use elite sample selection. The variance parameter must be reparameterised (e.g., $\nu = e^\eta$) to prevent it from going negative.

Information-Geometric Optimization (Section 8.7)

IGO applies the *natural gradient* — the ordinary gradient pre-multiplied by the inverse Fisher information matrix. This makes the updates invariant to reparameterisation of the distribution and to order-preserving transformations of f . In practice, raw function values are replaced by rank-based weights. For a Gaussian proposal, the natural gradient has a particularly clean closed form.

8.9 Chapter Summary V

CMA-ES (Section 8.8)

CMA-ES extends IGO by separately tracking a global step size σ and a shape matrix $\mathbf{C}$. Two “evolution paths” — running averages of recent mean movements — accumulate directional information across iterations, enabling efficient learning of the covariance structure. Step size is adapted based on whether the evolution path length is consistent with random walk behaviour. CMA-ES is generally considered the state-of-the-art method for zeroth-order optimisation in $n = 10$ to $n = 100$ dimensions.

8.10 Exercises I

Exercise 8.1: Limitations of Mixture Proposals in CEM

We have seen that mixture proposal distributions can better capture multiple modes of the objective (e.g., multiple global minima separated in space). Why might their use in the cross-entropy method be limited in practice?

Solution: The cross-entropy method requires fitting the proposal distribution to the elite samples at every iteration using maximum likelihood estimation (MLE). For a single Gaussian, MLE has an analytic closed form (the sample mean and covariance). Unfortunately, no known analytic solutions exist for fitting multivariate Gaussian *mixture* distributions. The standard alternative is the Expectation–Maximisation (EM) algorithm, which is itself an iterative procedure requiring many iterations to converge at each outer CEM iteration. This nested optimisation is computationally expensive and makes mixture proposals impractical for CEM in most real applications.

8.10 Exercises II

Exercise 8.2: Effect of Elite Fraction in CEM

In the cross-entropy method, what is the potential effect of using an elite sample size m_{elite} that is very close to the total sample size m ?

Solution: If $m_{\text{elite}}/m \rightarrow 1$ (elite fraction approaches 100%), almost all samples are considered “elite” regardless of their objective values. The refitted Gaussian will closely match the current proposal distribution, because nearly all samples are used in the MLE update. There is no strong selection pressure toward better regions of the design space, so the mean barely moves and the covariance barely shrinks. The result is extremely slow convergence: the algorithm makes negligible progress per iteration. This is analogous to natural selection with no selection pressure — all individuals survive equally, so the population does not evolve toward fitter traits.

8.10 Exercises III

Exercise 8.3: Variance Instability in PDD

Consider proposal distribution descent with $\mathcal{N}(\mu, \nu)$, with the mean sitting at the optimum: $\mu = x^*$. Show that the gradient of J with respect to ν (nu, the variance) is positive, which means gradient descent decreases ν toward zero. Explain why a fixed step size α (alpha) poses a numerical risk as $\nu \rightarrow 0$.

Solution: Near the optimum, approximate $f(x) \approx \frac{1}{2}h(x - x^*)^2$ where $h = f''(x^*) > 0$ (h is the second derivative, which is positive because x^* is a minimum). Substituting into the expected objective:

$$J(\mu, \nu) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \nu)} \left[\frac{1}{2}h(x - \mu)^2 \right] = \frac{1}{2}h\nu$$

Therefore $\nabla_{\nu} J = \frac{1}{2}h > 0$. Gradient descent on ν applies the update $\nu \leftarrow \nu - \alpha \cdot \frac{1}{2}h$, which decreases ν each iteration. Eventually ν reaches zero and then goes negative, which is invalid (a variance must be non-negative).

Moreover, as $\nu \rightarrow 0$, the score function $\nabla_{\nu} \log \mathcal{N} = \frac{(x-\mu)^2 - \nu}{2\nu^2}$ grows as $O(1/\nu^2)$, making the Monte Carlo gradient estimate extremely noisy for any finite sample size m .

The fix is to reparameterise: let $\nu = e^{\eta}$ (e to the power eta), so that ν is always positive. We then update η instead of ν directly. The chain rule gives $\nabla_{\eta} J = (\nabla_{\nu} J) \cdot \nu$, and since $\nu = e^{\eta} > 0$ always, there is no risk of the variance becoming negative.

Conclusion & Key Takeaways I

Chapter 8 has covered a broad spectrum of stochastic optimisation methods, unified by a common theme: using randomness not as a nuisance to be eliminated, but as a deliberate tool for exploration and robustness.

Conclusion & Key Takeaways II

Five Unifying Themes of Chapter 8

- 1 **Randomness is a feature, not a bug.** Every method in this chapter intentionally introduces randomness — either by perturbing the gradient (noisy descent, MeZO), by probing random directions (MADS), by controlling the probability of accepting worse solutions (SA), or by sampling from a distribution that evolves over time (CEM, PDD, IGO, CMA-ES). This randomness provides the essential ability to escape local minima.
- 2 **Temperature as an exploration-exploitation knob.** Simulated annealing provides the clearest illustration of a principle that runs through all distribution-based methods: early in the search, we want broad exploration; later, we want focused exploitation. The temperature schedule (or the shrinking variance of the proposal distribution) implements this transition automatically.
- 3 **Distribution-based methods turn optimisation into inference.** CEM, PDD, IGO, and CMA-ES all maintain a probability distribution over the design space and update it to concentrate near good solutions. This reframes optimisation as the question: “What distribution best describes where the minimum is likely to be?”
- 4 **The natural gradient provides invariance.** IGO and CMA-ES use the Fisher information matrix to correct for the curvature of the distribution manifold, making updates invariant to reparameterisation and to the absolute scale of f . This invariance is what makes CMA-ES so robust on ill-conditioned problems where the objective varies at very different rates in different directions.